



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 6: Các mẫu thiết kế tạo dựng

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Nội dung của chương này tập trung vào các mẫu thiết kế:

- Mẫu Factory method
- Mẫu Singleton
- Mẫu Abstract factory
- Mẫu Builder
- Mẫu Prototype

| 6.1 MẪU THIẾT KẾ FACTORY METHOD

6.1.1 Đặt vấn đề

- Trong các ngôn ngữ lập trình hướng đối tượng như Java, các lớp con trong cây phân cấp lớp có thể cài đặt đè phương thức lớp cha để cung cấp các kiểu chức năng khác nhau cho cùng tên phương thức.
- Khi một đối tượng ứng dụng biết chính xác chức năng cần thiết, nó sẽ khởi tạo trực tiếp lớp nào cung cấp chức năng đó. Tuy nhiên, có những tình huống mà đối tượng ứng dụng chỉ biết cần phải truy nhập lớp bên trong cây phân cấp nhưng không biết chính xác chọn lớp con nào.
- Như vậy, cần phải cài đặt một quy tắc chọn lớp dựa vào một số yếu tố như trạng thái của ứng dụng đang chạy, cấu hình...

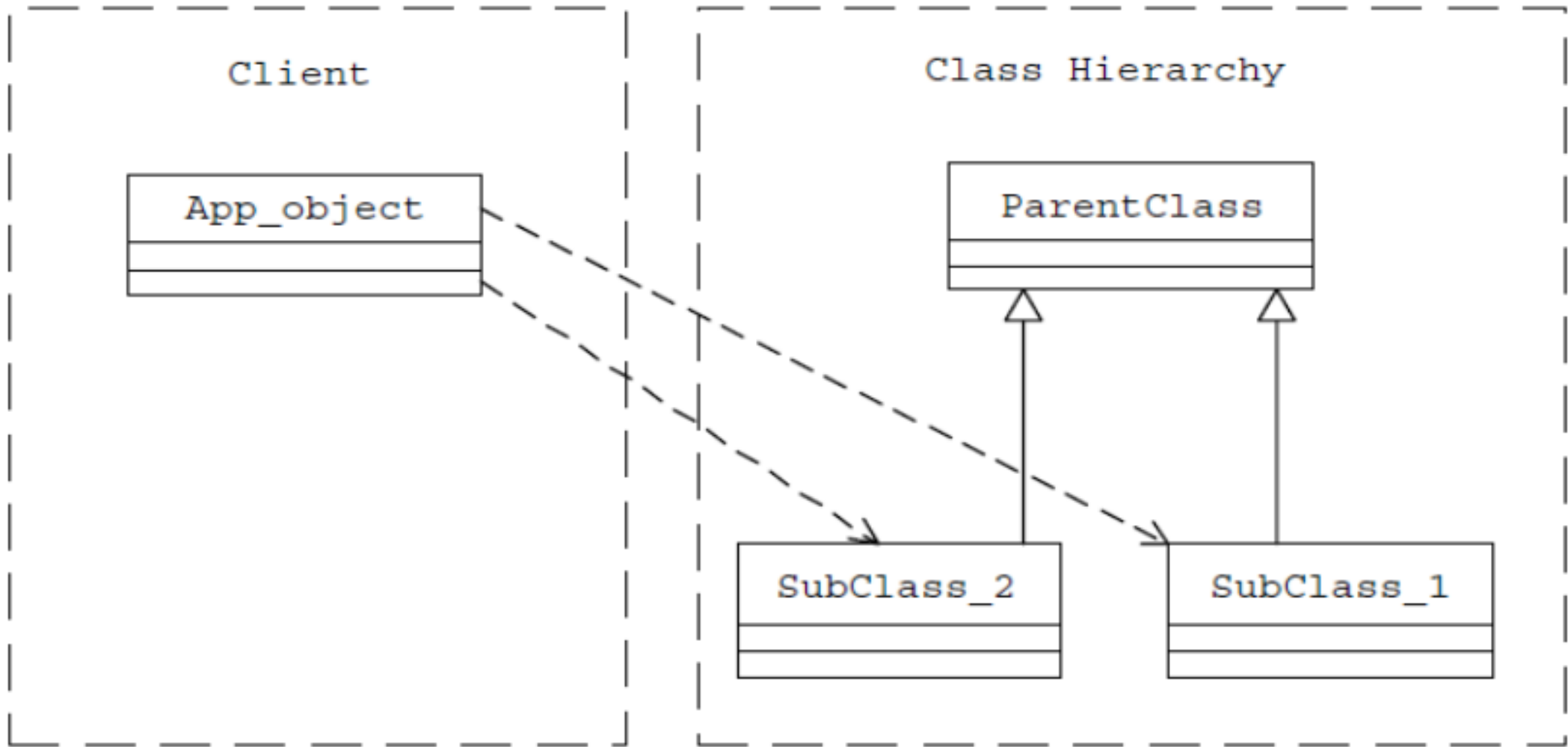
| MẪU THIẾT KẾ FACTORY METHOD

- Cách làm như vậy có thể dẫn đến nhiều bất lợi:
 - Gây nên sự kết nối chặt giữa đối tượng ứng dụng và cây phân cấp dịch vụ.
 - Khi quy tắc chọn lớp thay đổi thì mọi đối tượng ứng dụng cũng thay đổi theo.
 - Quy tắc chọn lớp đòi hỏi đối tượng ứng dụng phải biết đầy đủ về sự tồn tại và chức năng của mỗi lớp nên cài đặt có thể có những câu lệnh điều kiện bất hợp lý.

| 6.1.2 CẤU TRÚC MẪU

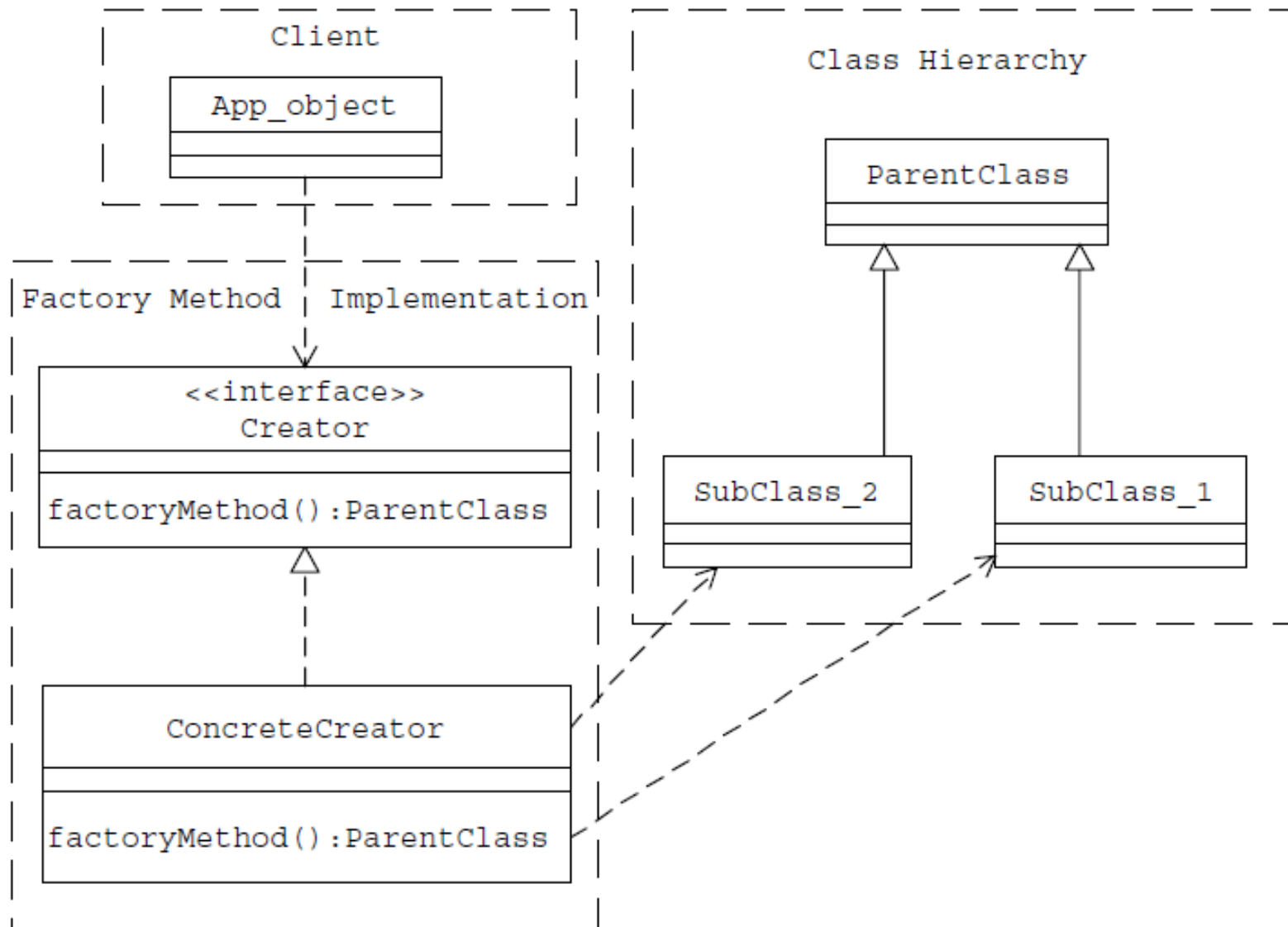
- Mẫu thiết kế **Factory method** có thể khắc phục nhược điểm trên bằng cách đóng gói chức năng yêu cầu và khởi tạo cũng như chọn lớp thích hợp bằng một phương thức gọi là **factoryMethod**.
- Phương thức này cũng được định nghĩa như một phương thức trong một lớp:
 - Chọn lớp thích hợp từ cây phân cấp lớp dựa vào ngữ cảnh ứng dụng và những yếu tố khác.
 - Khởi tạo lớp đã chọn và trả lại một thể hiện của kiểu lớp cha.

| CẤU TRÚC MẪU



Đối tượng client truy nhập trực tiếp cây phân cấp dịch vụ

| CẤU TRÚC MẪU



Đối tượng client truy nhập cây phân cấp dịch vụ qua factoryMethod

I CẤU TRÚC MẪU

Trong hình trên, nhận xét:

- **factoryMethod** trả lại thể hiện lớp đã chọn như đối tượng của kiểu lớp cha nên đối tượng ứng dụng không cần biết sự tồn tại của các lớp trong phân cấp.
- Một cách đơn giản để thiết kế **factoryMethod** là tạo ra một lớp trừu tượng **abstract** hay một giao tiếp **interface Creator** chỉ để khai báo **factoryMethod()**. Khi đó một lớp con **ConcreteCreator** sẽ được thiết kế để cài đặt toàn bộ **factoryMethod()**.
- Ta cũng có thể làm cách khác là cài đặt lớp **Creator** cụ thể với cài đặt mặc định **factoryMethod()** trong đó. Các lớp con khác của lớp **Creator** sẽ cài đè lên quy tắc chọn lớp cụ thể **factoryMethod()**.

| 6.1.3 CÁC TÌNH HUỐNG ÁP DỤNG FACTORY METHOD

Sau đây là một số tình huống có thể áp dụng **Factory Method**:

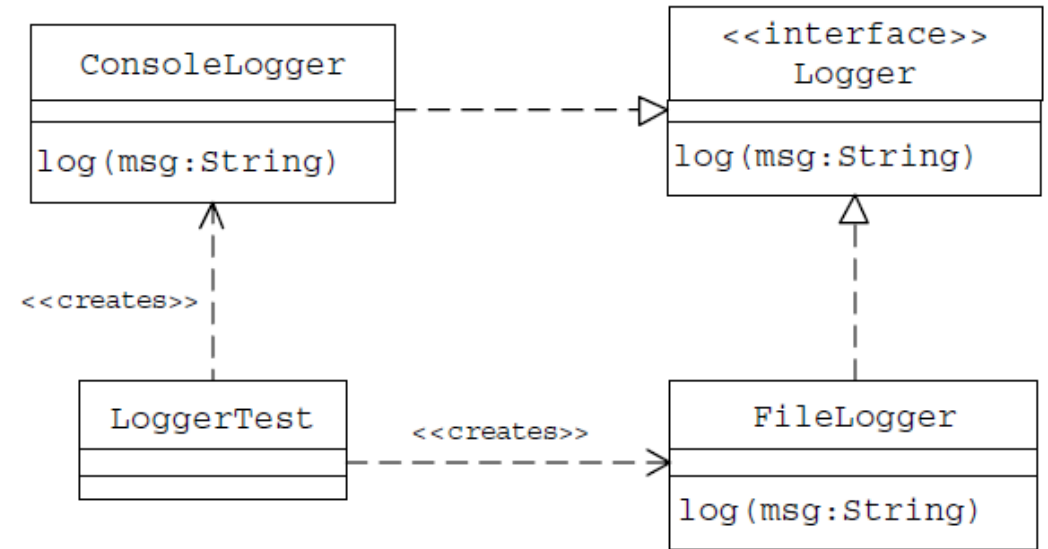
- Khi muốn tạo ra một framework để có thể mở rộng sau này, việc sử dụng mẫu factory method sẽ cho phép quyết định mềm dẻo khi chỉ ra loại đối tượng nào được tạo ra.
- Khi muốn một lớp con, mở rộng từ một lớp cha, quyết định lại đối tượng được khởi tạo.
- Khi ta biết khi nào thì khởi tạo một đối tượng nhưng không biết loại đối tượng nào được khởi tạo.
- Khi ta cần một vài khai báo chồng phương thức khởi tạo với danh sách các tham số như nhau, điều mà Java không cho phép. Thay vì điều đó ta sử dụng **các Factory Method** với các tên khác nhau.
- **Factory Method** thường được cài đặt cùng với **Abstract Factory** và **Prototype** và sẽ được trình bày sau. Khi đó lớp chứa **Factory Method** thường được gọi là **Template Method**.

| 6.1.4 VÍ DỤ FACTORY METHOD

- Thiết kế chức năng đưa ra log các thông điệp trong một ứng dụng. Log thông điệp thích hợp vào đúng thời điểm cần thiết đem lại nhiều ích lợi để bắt lỗi và theo dõi ứng dụng. Vì chức năng log thông điệp có thể cần đến cho nhiều client khác nhau, nên có thể cài đặt chức năng này trong một lớp **interface Logger**. Khi đó, các lớp cài đặt của **Logger** có thể xử lý để đưa ra thông điệp dưới dạng khác nhau cho các phương tiện khác nhau.

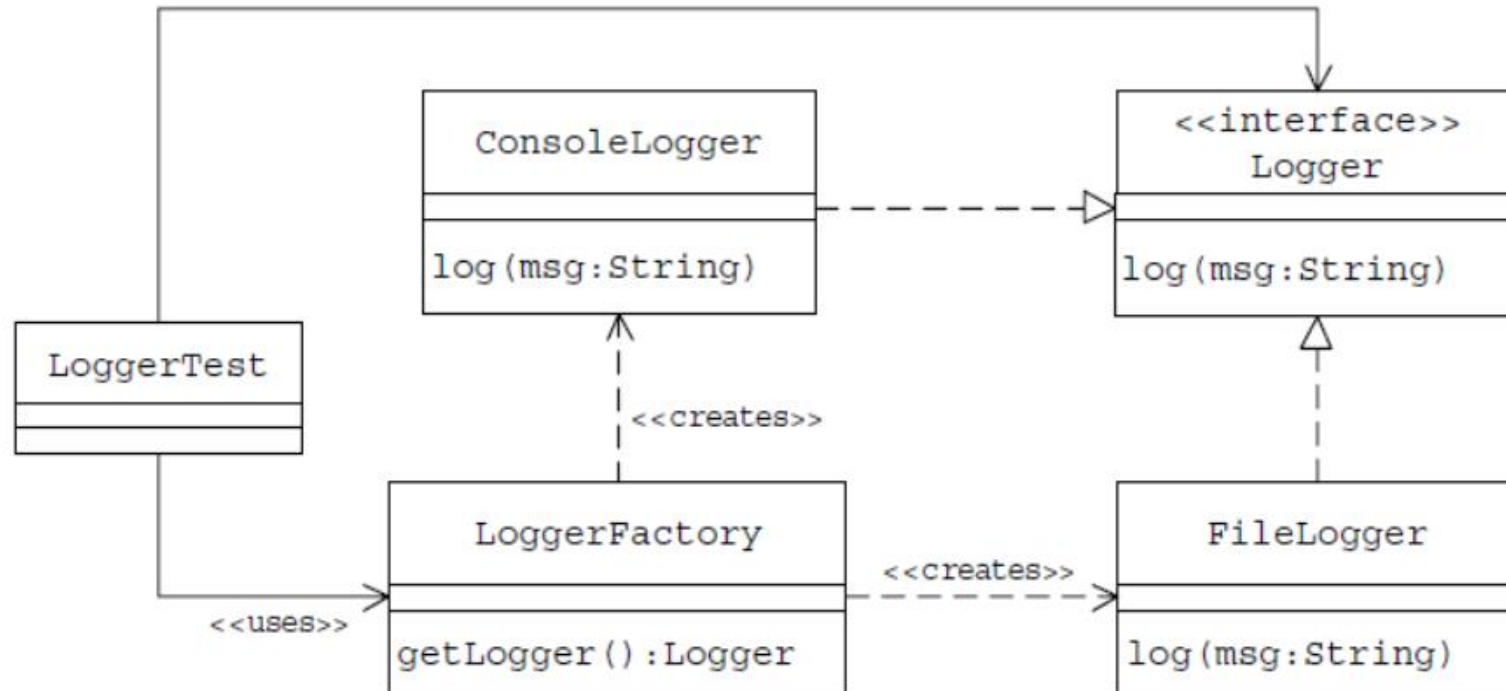
| Ví dụ Factory Method

- Cài đặt **FileLogger** có chức năng lưu thông điệp đến vào file log; **ConsoleLogger** hiển thị thông điệp trên màn hình.
- Hãy xét xem một đối tượng ứng dụng **LoggerTest** sử dụng các dịch vụ cung cấp bởi các cài đặt này sẽ như thế nào.
- Làm sao đối tượng biết khởi tạo **FileLogger** hay **ConsoleLogger**?
 - > Sử dụng file thuộc tính **logger.properties** cung cấp thêm thông tin để lựa chọn và khởi tạo lớp con.



| Ví dụ Factory Method

- Khi áp dụng mẫu **Factory method**, việc chọn và khởi tạo **Logger** thích hợp có thể được đóng gói bên trong phương thức **getLogger** trong lớp **LoggerFactory** như trong hình.



| Ví dụ Factory Method

```
//Logger
public interface Logger {
    public void log(String msg);
}

//FileLogger
public class FileLogger implements Logger {
    public void log(String msg) {
        FileUtil futil = new FileUtil();
        futil.writeToFile("log.txt", msg, true, true);
    }
}

//ConsoleLogger
public class ConsoleLogger implements Logger {
    public void log(String msg) {
        System.out.println(msg);
    }
}
```

| Ví dụ Factory Method

```
//LoggerFactory
public class LoggerFactory {
    public boolean isFileLoggingEnabled() {
        Properties p = new Properties();
        try {
            p.load(ClassLoader.getResourceAsStream("Logger.properties"));
            String fileLoggingValue = p.getProperty("FileLogging");
            if (fileLoggingValue.equalsIgnoreCase("ON") == true)
                return true;
            else
                return false;
        } catch (IOException e) {
            return false;
        }
    }
    //Factory Method
    public Logger getLogger() {
        if (isFileLoggingEnabled()) {
            return new FileLogger();
        } else {
            return new ConsoleLogger();
        }
    }
}
```

| Ví dụ Factory Method

```
//LoggerTest
public class LoggerTest {
    public static void main(String[] args) {
        LoggerFactory factory = new LoggerFactory();
        Logger logger = factory.getLogger();
        logger.log("A Message to Log");
    }
}
```

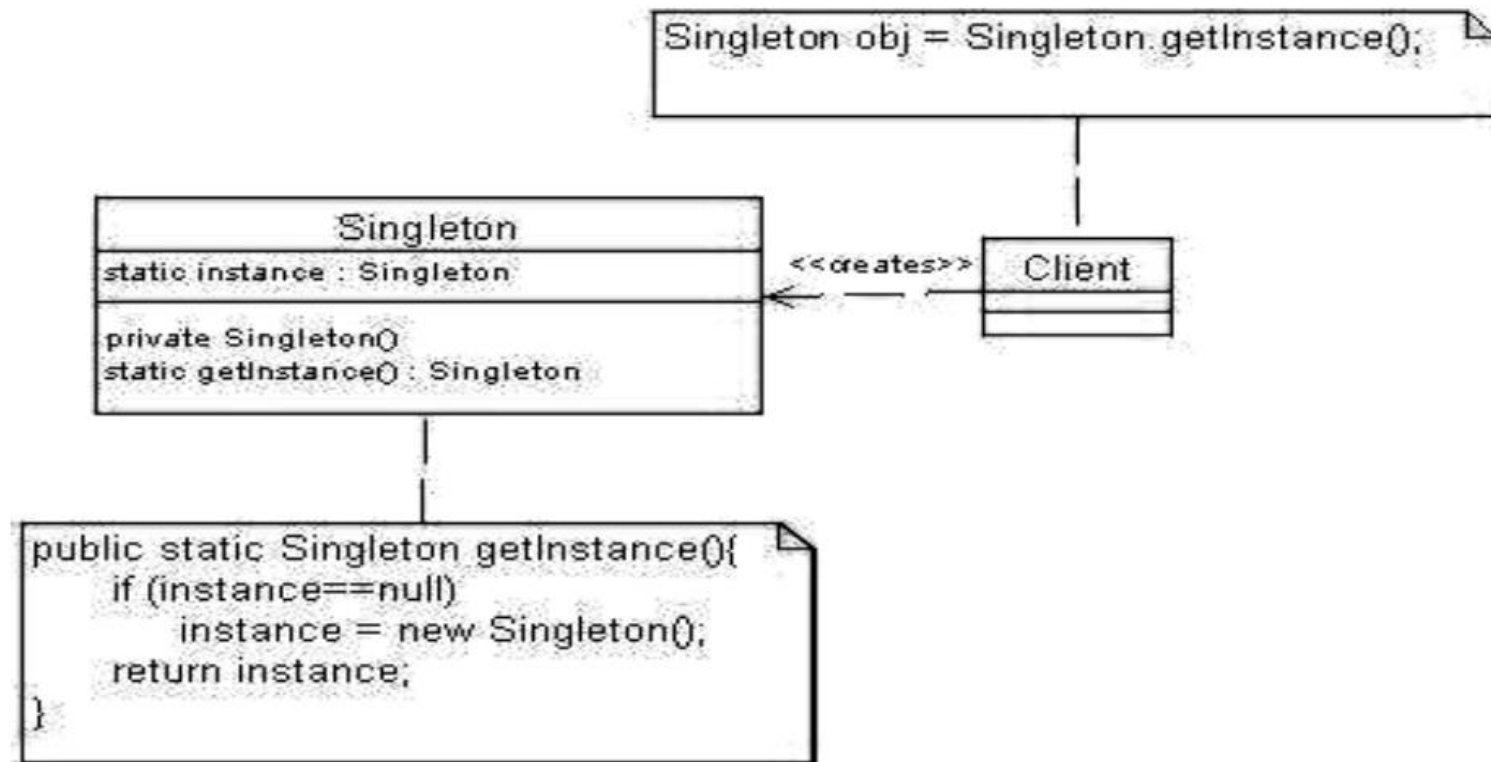
| 6.2 MẪU THIẾT KẾ SINGLETON

6.2.1 Đặt vấn đề

- Hãy xét tình huống một đối tượng có chức năng kết nối dữ liệu để cập nhật thông tin như thay đổi, thêm, xóa...Rõ ràng, khi đó chỉ có duy nhất một đối tượng trong vòng đời của ứng dụng này. Mẫu **Singleton** là lớp được sử dụng trong những trường hợp như thế để đảm bảo rằng có một và chỉ một thể hiện đối tượng.
- Lớp **Singleton** bảo đảm khởi tạo duy nhất bằng cách sử dụng tham chiếu đến chính nó.

| 6.2.1 Cấu trúc mẫu

- Biểu đồ lớp cho **Singleton** được thể hiện như trong hình, trong đó hàm **getInstance()** đặt ở dạng **static** và khởi tạo đối tượng trong chính lớp **Singleton**.



| 6.2.3 Tình huống áp dụng

- Mẫu singleton thông thường được cài đặt với các mẫu **abstract factory**, **builder**, **prototype**.

| 6.2.4 Ví dụ Singleton

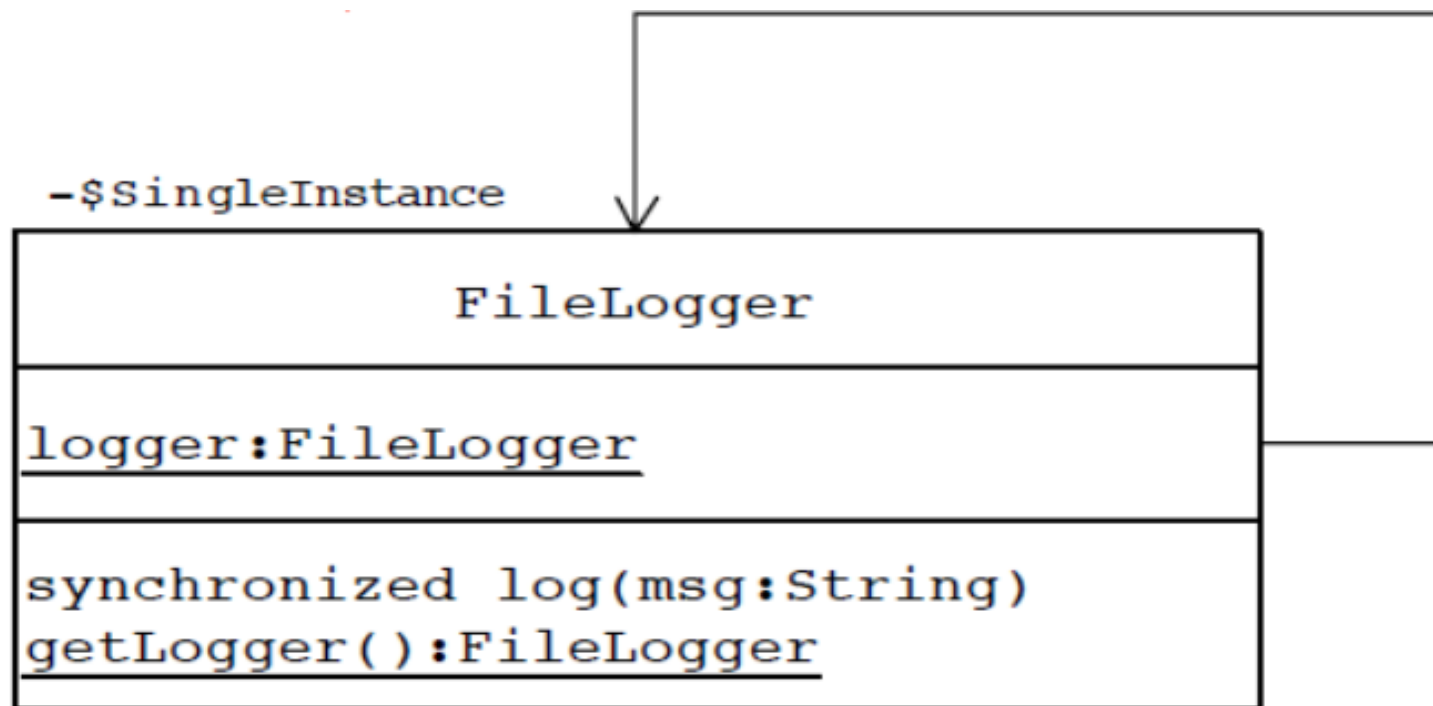
- Trở lại ví dụ trong phần trước khi thiết kế với mẫu **Factory method**. Cài đặt **FileLogger** của giao tiếp interface **Logger** nhằm chuyển thông điệp vào file **log.txt**. Nếu chỉ có một thể hiện vật lý của đối tượng đại diện thì sẽ tốt hơn. Trong một ứng dụng, khi những đối tượng client khác nhau truyền những thông điệp vào một file thì sẽ có khả năng là nhiều thể hiện của lớp **FileLogger** sử dụng bởi từng đối tượng client. Điều này có thể dẫn đến nhiều vấn đề do các đối tượng khác nhau đồng thời truy nhập vào cùng file.

| Ví dụ Singleton

- Một trong những giải pháp là duy trì một thể hiện của lớp **FileLogger** trong biến toàn cục của ứng dụng. Thể hiện này có thể được truy nhập bởi tất cả client bằng cách cung cấp cho nó một điểm truy nhập toàn cục duy nhất. Tuy nhiên, cách làm như vậy không thể giải quyết trọn vẹn vấn đề. Một là không ngăn ngừa được client tạo các thể hiện mới từ lớp **FileLogger**. Hai là nó cũng không tránh được nhiều luồng trong cùng client thực thi phương thức log.
- Một giải pháp khác là áp dụng khái niệm giám sát (đảm bảo không có hai luồng được phép truy nhập cùng một đối tượng tại cùng thời điểm) và khai báo phương thức **log(String)** là đồng bộ. Điều này tránh được nhiều luồng cùng thực thi một phương thức nhưng không ngăn các đối tượng client tạo nhiều thể hiện của lớp **FileLogger**.

| Ví dụ Singleton

- Ngoài việc cần khai báo tính đồng bộ của phương thức log, chúng ta cũng phải cần đảm bảo rằng chỉ tồn tại một thể hiện của **FileLogger** suốt trong vòng đời của ứng dụng. Để thực hiện được việc này, chúng ta sẽ thay đổi lớp **FileLogger** thành lớp **Singleton**.



| Ví dụ Singleton

```
//Singleton Filelogger class
public class Filelogger implements Logger{
    private static FileLogger logger;
    //Ngăn ngừa client sử dụng
    private FileLogger(){
    }
    public static Filelogger getFileLogger(){
        if (logger == null){
            logger = new FileLogger();
        }
        return logger;
    }
    public synchronized void log(String msg){
        FileUtile futil = new FileUtil();
        Futil.writeToFile("log.txt", msg, true, true);
    }
}
```

Việc tạo **private** cho private **FileLogger()** là nhằm ngăn các đối tượng client tạo đối tượng **FileLogger** nhưng những phương thức khác trong **FileLogger** có thể truy nhập được.

| Ví dụ Singleton

```
//Lớp LoggerFactory
public class LoggerFactory {
    public boolean isFileLoggingEnabled() {
        Properties p = new Properties();
        try {
            p.load(ClassLoader.getResourceAsStream("Logger.properties"));
            String fileLoggingValue = p.getProperty("FileLogging");
            if (fileLoggingValue.equalsIgnoreCase("ON") == true)
                return true;
            else
                return false;
        } catch (IOException e) {
            return false;
        }
    }
    //Factory Method
    public Logger getLogger() {
        if (isFileLoggingEnabled()) {
            return Filelogger.getFileLogger();
        } else {
            return new ConsoleLogger();
        }
    }
}
```

| 6.3 MẪU THIẾT KẾ ABSTRACT METHOD

6.3.1 Đặt vấn đề

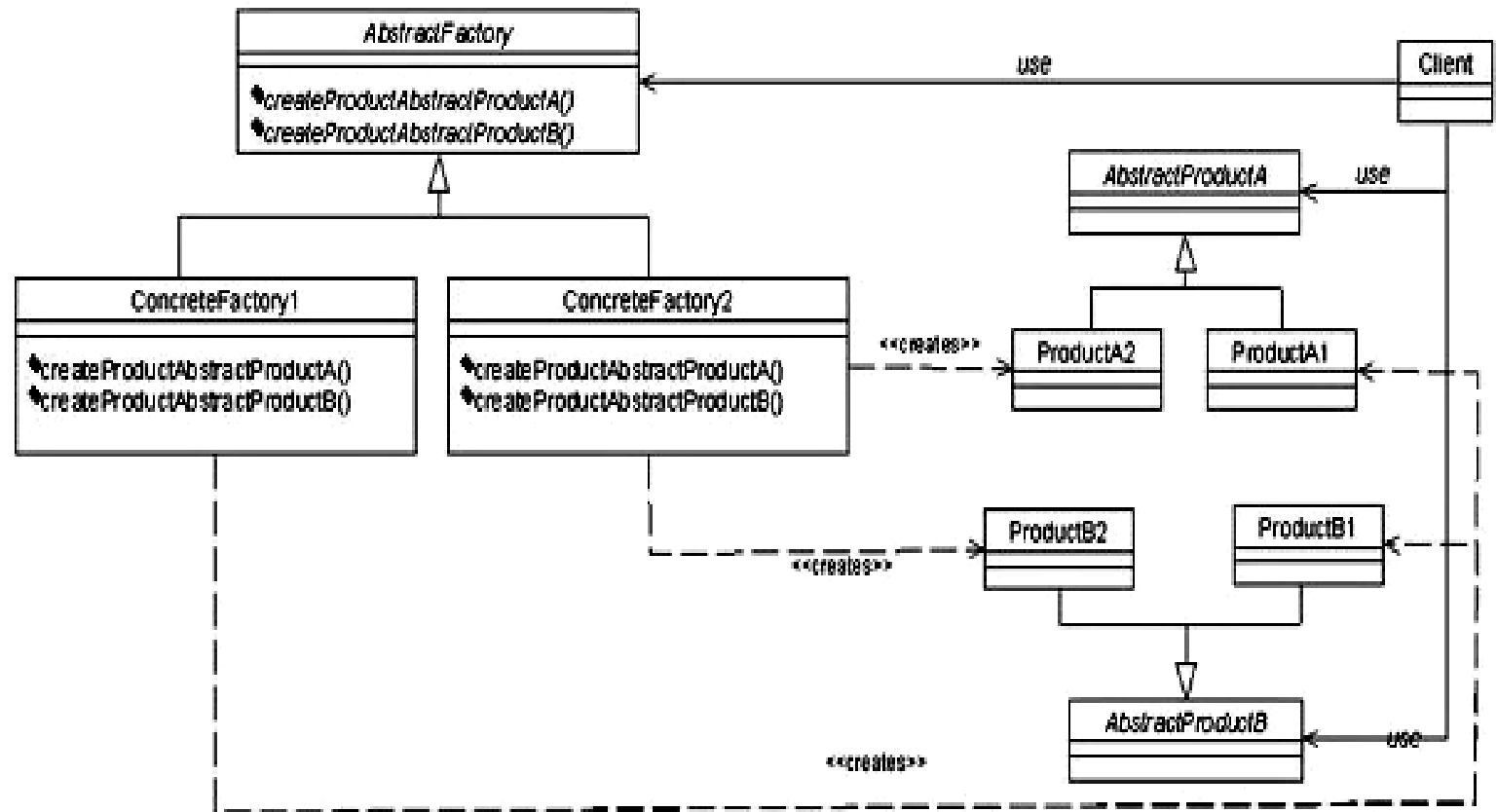
- Trong các hệ điều hành hay phần mềm có giao diện đồ họa, thường có một bộ công cụ cung cấp một giao diện người dùng dựa trên chuẩn “nhìn và cảm nhận”. Ví dụ, trong chương trình trình diễn tài liệu Powerpoint, có rất nhiều kiểu giao diện như vậy và cả những hành vi giao diện người dùng khác nhau được thể hiện ở đây như thanh cuộn tài liệu, cửa sổ, nút bấm, hộp soạn thảo...
- Nếu xem chúng là các đối tượng thì chúng có một số đặc điểm và hành vi khá giống nhau về mặt hình thức nhưng lại khác nhau về cách thực hiện.
- Chẳng hạn đối tượng nút bấm và cửa sổ, hộp soạn thảo có cùng các thuộc tính là chiều rộng, chiều cao, tọa độ...và có các phương thức là `Resize()`, `SetPosition()`... Tuy nhiên, các đối tượng này không thể gộp chung được vào một lớp vì các đối tượng ở đây dù có cùng giao diện nhưng cách thực hiện các hành vi tương ứng lại hoàn toàn khác nhau

| 6.3.1 Đặt vấn đề

- Vấn đề đặt ra là có thể xây dựng một lớp tổng quát chứa những điểm chung của các đối tượng này để từ đó có thể dễ dàng sử dụng lại. Trong chương trình Powerpoint, lớp **WidgetFactory** đã được xây dựng để các lớp của các đối tượng cửa sổ, nút bấm, hộp soạn thảo kế thừa từ lớp này.
- Mẫu **AbstractFactory** là một mẫu thiết kế nhằm cung cấp cho trình khách một giao tiếp interface để cho các đối tượng thuộc các lớp khác nhau nhưng có chung giao tiếp có thể hoạt động mà không phải trực tiếp làm việc với từng lớp con cụ thể.

| 6.3.2 Cấu trúc mẫu

- **AbstractFactory**: là lớp trừu tượng, tạo ra các đối tượng thuộc hai lớp trừu tượng là **AbstractProductA** và **AbstractProductB**.
- **ConcreteFactoryX**: là lớp kế thừa từ **AbstractFactory**, lớp này sẽ tạo ra một đối tượng cụ thể.
- **AbstractProduct**: là các lớp trừu tượng, các đối tượng cụ thể sẽ là các thể hiện của các lớp dẫn xuất từ lớp này.

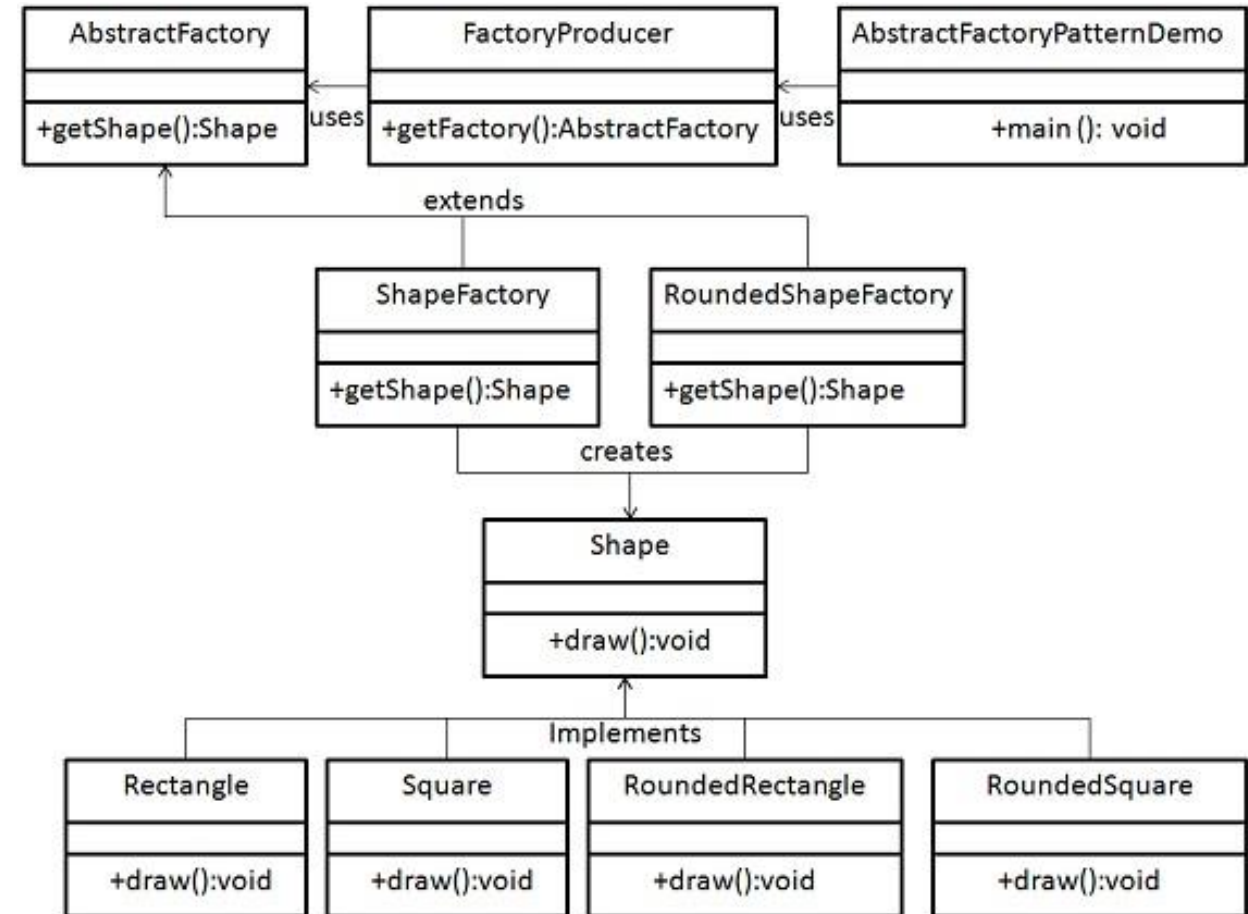


| 6.3.3 Tình huống áp dụng

- Ứng dụng sẽ được cấu hình với một hoặc nhiều họ sản phẩm và các đối tượng cần phải được tạo ra như là một tập hợp để có thể tương thích với nhau.
- Phía client không phụ thuộc vào việc những sản phẩm được tạo ra như thế nào.
- Chúng ta muốn cung cấp một tập các lớp và muốn thể hiện các ràng buộc, các mối quan hệ giữa chúng mà không phải là các thực thi của chúng.
- Mẫu **abstract factory** thường được cài đặt cùng với các mẫu **singleton**, **factory method** và đôi khi còn dùng với **prototype**.

| 6.3.4 Ví dụ Abstract Factory

- Chúng ta tạo một interface **Shape** và các lớp cụ thể để triển khai nó: **Rectangle**, **Square**, **RoundedRectangle**, **RoundedSquare**.
- Tạo tiếp lớp trừu tượng **AbstractFactory** và các lớp **ShapeFactory** và **RoundedFactory** là những lớp mở rộng của lớp này.
- Tạo lớp **FactoryProducer** để khởi tạo các lớp.
- Lớp **AbstractFactoryPatternDemo** là lớp main demo sử dụng **FactoryProducer** để lấy một đối tượng **AbstractFactory**, nó sẽ truyền vào tham số loại Shape (RECTANGLE / SQUARE) đến **AbstractFactory** để lấy loại đối tượng mà nó cần.



| Ví dụ Abstract Factory

Bước 1: Tạo interface Shape

Shape.java

```
public interface Shape {  
    void draw();  
}
```

Bước 2: Tạo các lớp cụ thể

RoundedRectangle.java

```
public class RoundedRectangle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("RoundedRectangle::draw().");  
    }  
}
```

RoundedSquare.java

```
public class RoundedSquare implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("RoundedSquare::draw().");  
    }  
}
```

Rectangle.java

```
public class Rectangle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Rectangle::draw().");  
    }  
}
```

Bước 3: Tạo lớp trừu tượng AbstractFactory

AbstractFactory.java

```
public abstract class AbstractFactory {  
    abstract Shape getShape(String shapeType) ;  
}
```

| Ví dụ Abstract Factory

Bước 4: Tạo các lớp mở rộng từ AbstractFactory để khởi tạo đối tượng của lớp cụ thể dựa trên thông tin cung cấp.

ShapeFactory.java

```
public class ShapeFactory extends AbstractFactory {
    @Override
    public Shape getShape(String shapeType){
        if(shapeType.equalsIgnoreCase("RECTANGLE")){
            return new Rectangle();
        }else if(shapeType.equalsIgnoreCase("SQUARE")){
            return new Square();
        }
        return null;
    }
}
```

RoundedShapeFactory.java

```
public class RoundedShapeFactory extends AbstractFactory {
    @Override
    public Shape getShape(String shapeType){
        if(shapeType.equalsIgnoreCase("RECTANGLE")){
            return new RoundedRectangle();
        }else if(shapeType.equalsIgnoreCase("SQUARE")){
            return new RoundedSquare();
        }
        return null;
    }
}
```

| Ví dụ Abstract Factory

Bước 5: Tạo lớp FactoryProducer trả về lớp mở rộng của AbstractFactory.

FactoryProducer.java

```
public class FactoryProducer {  
    public static AbstractFactory getFactory(boolean rounded){  
        if(rounded){  
            return new RoundedShapeFactory();  
        }else{  
            return new ShapeFactory();  
        }  
    }  
}
```

| Ví dụ Abstract Factory

Bước 6: Sử dụng FactoryProducer lấy AbstractFactory để lấy các lớp cụ thể bằng cách đưa vào loại Shape.

```
public class AbstractFactoryPatternDemo {  
    public static void main(String[] args) {  
        //get shape factory  
        AbstractFactory shapeFactory = FactoryProducer.getFactory(false);  
        //get an object of Shape Rectangle  
        Shape shape1 = shapeFactory.getShape("RECTANGLE");  
        //call draw method of Shape Rectangle  
        shape1.draw();  
        //get an object of Shape Square  
        Shape shape2 = shapeFactory.getShape("SQUARE");  
        //call draw method of Shape Square  
        shape2.draw();  
        //get shape factory  
        AbstractFactory shapeFactory1 = FactoryProducer.getFactory(true);  
        //get an object of Shape Rectangle  
        Shape shape3 = shapeFactory1.getShape("RECTANGLE");  
        //call draw method of Shape Rectangle  
        shape3.draw();  
        //get an object of Shape Square  
        Shape shape4 = shapeFactory1.getShape("SQUARE");  
        //call draw method of Shape Square  
        shape4.draw();  
    }  
}
```

Bước 7: Xem kết quả

```
Rectangle::draw().  
Square::draw().  
RoundedRectangle::draw().  
RoundedSquare::draw().
```

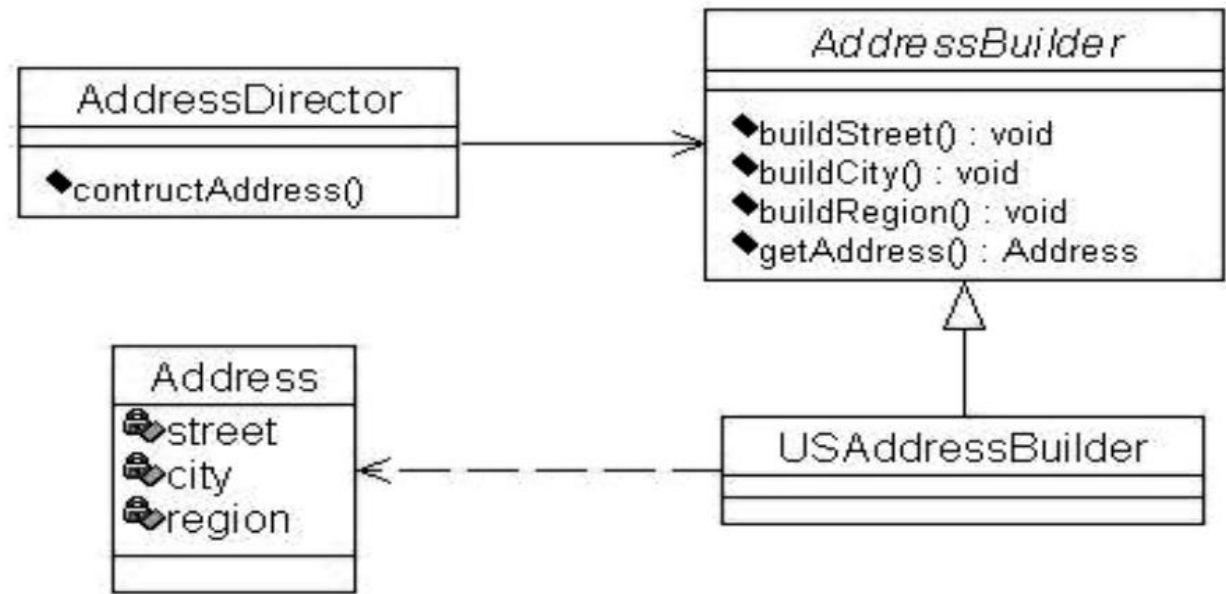

| 6.4 MẪU THIẾT KẾ BUILDER

6.4.1 Đặt vấn đề

- Các ứng dụng lớn thường có nhiều chức năng phức tạp và giao diện đồ sộ. Khi đó, việc khởi tạo ứng dụng thường gặp nhiều khó khăn. Nếu dồn tất cả công việc này cho một hàm khởi tạo thì sẽ rất khó kiểm soát và hơn nữa không phải lúc nào các thành phần ứng dụng cũng được khởi tạo một cách đồng bộ.
- Vì có những thành phần được tạo ra vào lúc dịch chương trình nhưng cũng có thành phần được tạo ra tùy theo từng yêu cầu của người dùng hay ngữ cảnh của ứng dụng. Do vậy, cách tốt nhất là giao công việc này cho một đối tượng chịu trách nhiệm khởi tạo và phân chia việc khởi tạo ứng dụng để có thể tiến hành khởi tạo một cách riêng biệt ở các hoàn cảnh khác nhau.
- **Builder** là mẫu thiết kế được tạo ra để chia công việc khởi tạo một đối tượng phức tạp thành khởi tạo các đối tượng riêng rẽ để từ đó có thể tiến hành khởi tạo đối tượng trong các ngữ cảnh khác nhau.

| 6.4.2 Cấu trúc mẫu

- **AddressDirector** là lớp tạo ra đối tượng **Address**.
- **AddressBuilder** là lớp trừu tượng cho phép tạo ra một đối tượng **Address** bằng các phương thức khởi tạo từng thành phần của **Address**.
- **USAddressBuilder** là lớp tạo ra các **Address**. **USAddressBuilder** sẽ tạo ra địa chỉ theo chuẩn của USA.



| 8.4.3 Tình huống áp dụng

- Mẫu **Builder** thường được cài đặt cùng với các mẫu như **Abstract Factory**. Ý nghĩa quan trọng của **Abstract Factory** là tạo ra một dòng các đối tượng dẫn xuất (cả đơn giản và phức tạp).
- Ngoài ra **Builder** còn thường được cài đặt kèm với mẫu **Composite**. Mẫu **Composite** thường là những gì mà **Builder** tạo ra.

| 6.4.4 Ví dụ mẫu Builder

- Xét lớp `Address` với các thành phần như sau: `Street`, `City` và `Region`.
- Ta phân rã việc khởi tạo một đối tượng `Address` thành các phần: `buildStreet`, `buildCity` và `buildRegion`.

| Ví dụ mẫu Builder

```
class Address{
    private String street;
    private String city;
    private String region;

    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public String getRegion() {
        return region;
    }
    public void setRegion(String region) {
        this.region = region;
    }
    public String getStreet() {
        return street;
    }
    public void setStreet(String street) {
        this.street = street;
    }
}
```

```
abstract class AddressBuilder{
    abstract public void buildStreet(String street){}
    abstract public void buildCity(String city){}
    abstract public void buildRegion(String region){}
}

class USAddressBuilder extends AddressBuilder {
    private Address add;
    public void buildStreet(String street){
        add.setStreet(street);
    }
    public void buildCity(String city){
        add.setCity(city);
    }
    public void buildRegion(String region){
        add.setRegion(region);
    }
    public Address getAddress(){
        return add;
    }
}
```

| Ví dụ mẫu Builder

```
class AddressDirector{  
    public void Construct(AddressBuilder builder, String street,  
        String city, String region){  
        builder.buildStreet(street);  
        builder.buildCity(city);  
        builder.buildRegion(region);  
    }  
}  
  
class Client{  
    AddressDirector director = new AddressDirector();  
    USAddressBuilder b = new USAddressBuilder();  
    director.Construct(b, "Street 01", "City 01", "Region 01");  
    Address add = b.getAddress();  
}
```

| 6.5 MẪU THIẾT KẾ PROTOTYPE

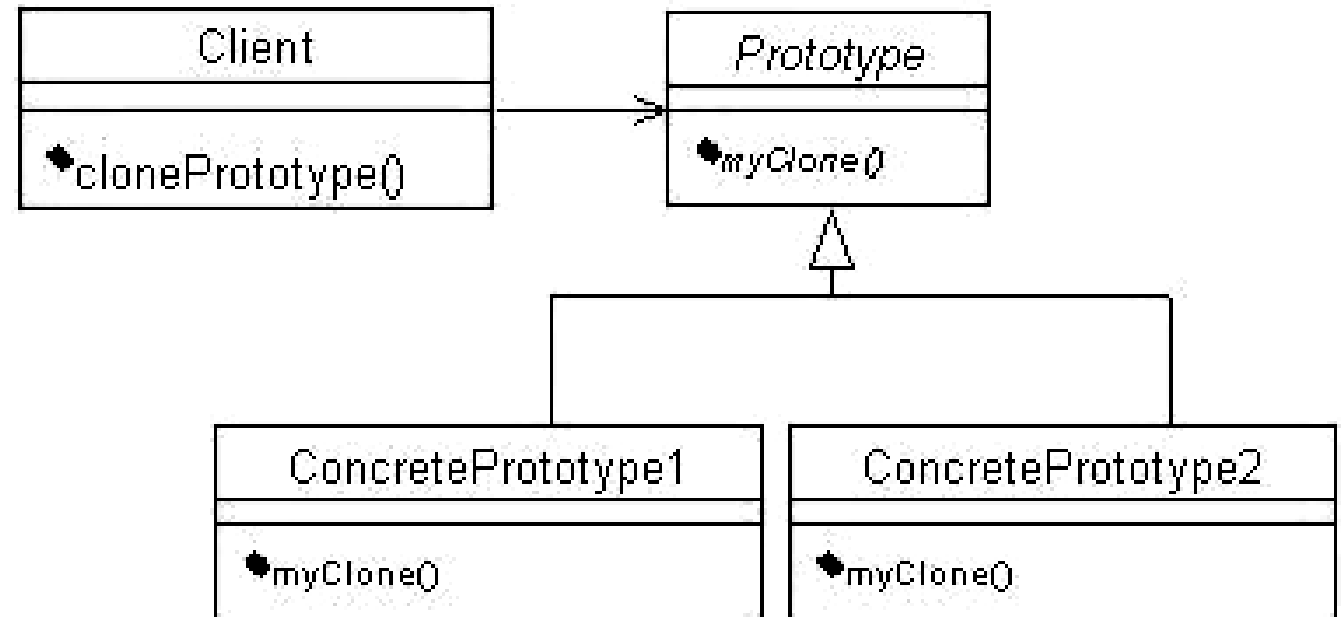
6.5.1 Đặt vấn đề

- Cả hai mẫu **Factory Method** và **Abstract Factory** cho phép tiến trình độc lập tạo ra các đối tượng. **Prototype** cũng giải quyết vấn đề tương tự nhưng theo cách khác linh hoạt hơn.
- **Prototype** là mẫu thiết kế có thể chỉ định một đối tượng đặc biệt để khởi tạo. Nó sử dụng một thể hiện cơ bản rồi sau đó sao chép ra các đối tượng khác từ mẫu đối tượng này.

| 6.5.2 Cấu trúc mẫu

Trong đó:

- **Prototype** là lớp trừu tượng cài đặt phương thức **myClone()** – một phương thức copy bản thân đối tượng đã tồn tại.
- **ConcretePrototype1** và **ConcretePrototype2** là các lớp kế thừa từ lớp **Prototype**.

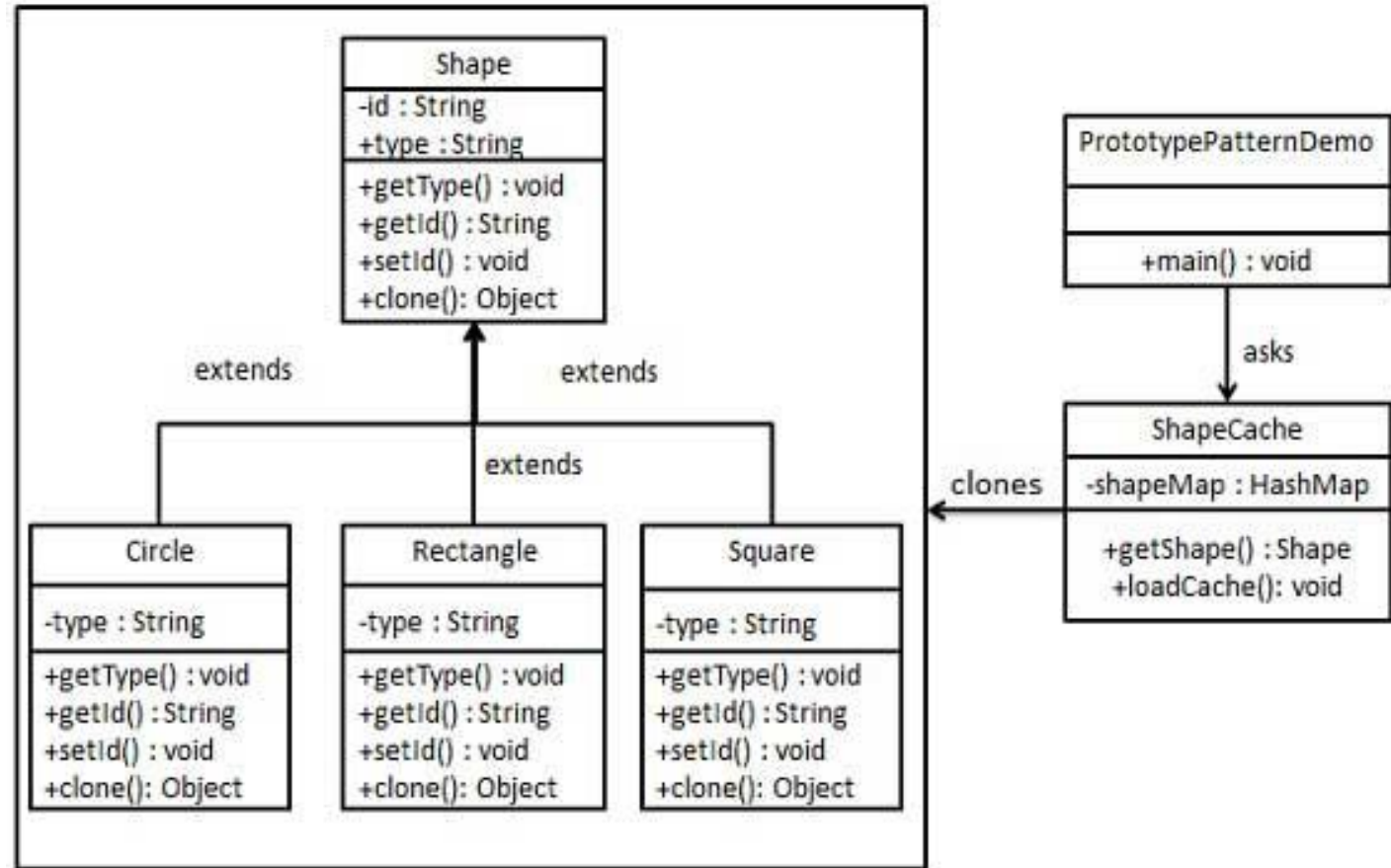


| 6.5.3 Tình huống áp dụng

- Sử dụng mẫu thiết kế **Prototype** khi muốn khởi tạo một đối tượng bằng cách sao chép từ một đối tượng đã tồn tại.
- Mẫu này được sử dụng khi tạo đối tượng trực tiếp sẽ tốn tài nguyên hơn là sao chép nó.
- Mặc dù đôi khi đối chọi nhau, mẫu **Prototype** và **Abstract Factory** vẫn có thể kết hợp với nhau.

| 6.5.4 Ví dụ mẫu Prototype

- Chúng ta tạo một lớp trừu tượng **Shape** và các lớp cụ thể mở rộng từ lớp **Shape**: **Circle**, **Rectangle**, **Square**.
- Lớp **ShapeCache** lưu trữ đối tượng shape trong **Hashtable** và trả về bản sao của chúng khi có yêu cầu.
- Lớp **PrototypePatternDemo** sử dụng lớp **ShapeCache** để lấy đối tượng **Shape**.



| Ví dụ mẫu Prototype

Bước 1: Tạo lớp Shape implement Cloneable interface

```
public abstract class Shape implements Cloneable {  
    private String id;  
    protected String type;  
    abstract void draw();  
  
    public String getType() {  
        return type;  
    }  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
    public Object clone() {  
        Object clone = null;  
        try {  
            clone = super.clone();  
        } catch (CloneNotSupportedException e) {  
            e.printStackTrace();  
        }  
        return clone;  
    }  
}
```

| Ví dụ mẫu Prototype

Bước 2: Tạo các lớp mở rộng từ Shape

Rectangle.java

```
public class Rectangle extends Shape {  
  
    public Rectangle(){  
        type = "Rectangle";  
    }  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Rectangle::draw() method.");  
    }  
}
```

Circle.java

```
public class Circle extends Shape {  
  
    public Circle(){  
        type = "Circle";  
    }  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Circle::draw() method.");  
    }  
}
```

Square.java

```
public class Square extends Shape {  
  
    public Square(){  
        type = "Square";  
    }  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Square::draw() method.");  
    }  
}
```

| Ví dụ mẫu Prototype

Bước 3: Tạo một lớp để lấy lớp cụ thể từ database và lưu chúng vào Hashtable.

```
import java.util.Hashtable;

public class ShapeCache {
    private static Hashtable<String, Shape> shapeMap = new Hashtable<String, Shape>();
    public static Shape getShape(String shapeId) {
        Shape cachedShape = shapeMap.get(shapeId);
        return (Shape) cachedShape.clone();
    }
    public static void loadCache() {
        Circle circle = new Circle();
        circle.setId("1");
        shapeMap.put(circle.getId(), circle);

        Square square = new Square();
        square.setId("2");
        shapeMap.put(square.getId(), square);

        Rectangle rectangle = new Rectangle();
        rectangle.setId("3");
        shapeMap.put(rectangle.getId(), rectangle);
    }
}
```

| Ví dụ mẫu Prototype

Bước 4: PrototypePatternDemo sử dụng ShapeCache để lấy bản sao của shape trong Hashtable.

```
public class PrototypePatternDemo {  
    public static void main(String[] args) {  
        ShapeCache.loadCache();  
  
        Shape clonedShape = (Shape) ShapeCache.getShape("1");  
        System.out.println("Shape : " + clonedShape.getType());  
  
        Shape clonedShape2 = (Shape) ShapeCache.getShape("2");  
        System.out.println("Shape : " + clonedShape2.getType());  
  
        Shape clonedShape3 = (Shape) ShapeCache.getShape("3");  
        System.out.println("Shape : " + clonedShape3.getType());  
    }  
}
```

Kết quả

Shape : Circle
Shape : Square
Shape : Rectangle

| Bài tập thực hành chương 6

- Lập trình cài đặt và hoàn thiện ví dụ trong các mẫu thiết kế của bài học: Factory method, Singleton, Abstract factory, Builder, Prototype.
(Hoặc các ví dụ tương tự)

| Bài tập tính điểm chương 5&6

1. Hoàn thành 4 bài tập trong chương 5.
 2. Xây dựng các mẫu thiết kế **Factory**, **Singleton** cho hệ thống bán sách online.
 - Ghi chú: Chỉ thiết kế và cài đặt mẫu, không cần lập trình cả hệ thống.
 - Gợi ý: Liệt kê các chức năng cơ bản cần có của hệ thống và thiết kế các mẫu để đáp ứng các chức năng này.
- Hình thức:
 - Bài tập làm theo nhóm (tối đa 3 người).
 - Đóng cuốn để nộp, ghi rõ phân công nhiệm vụ từng bạn.
 - Hạn nộp bài: 19/04/2025